Zhi-Hua Zhou

Machine Learning

Translated by Shaowu Liu

The slides of this chapter are translated by Yun-Hao Cao

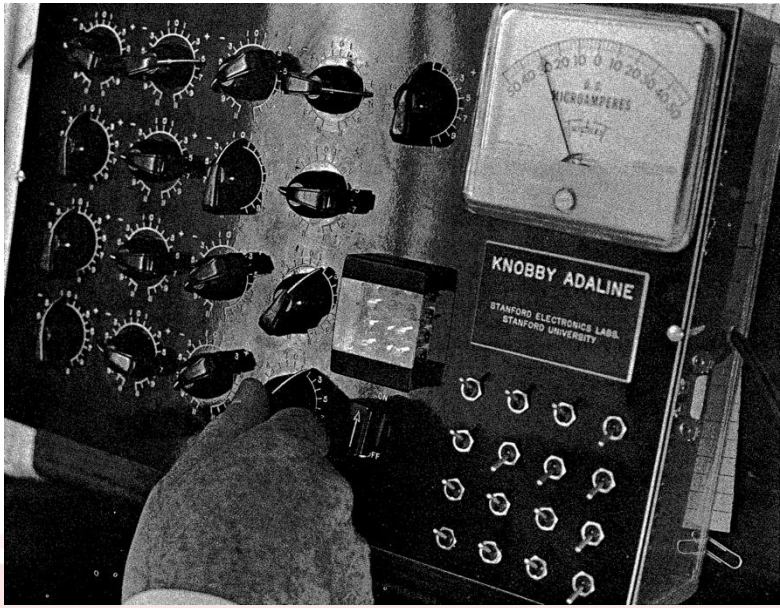
Springer

Chapter 5
Neural Networks

Neural network history

First stage

- In 1943, McCulloch and Pitts proposed the first neural model, i.e., M-P neuron model, and proved in principle that the artificial neural network can calculate any arithmetic and logical function.
- In 1949, Hebb published the book《The Organization of Behavior》, and proposed the mechanism of biological neuron learning, i.e., the Hebb learning rule
- In 1958, Rosenblatt proposed Perceptron and its learning rule
- In 1960, Widrow and Hoff proposed Adaline and the Least Mean Square (LMS) algorithm



Neural network history

First stage

- In 1969, Minsky and Papert published the book 《Perceptrons》, which pointed out that single-layer neural network cannot solve non-linear problems, and it is unknown whether it is possible to train multiple-layer networks. This conclusion directly pushed neural network research into an "ice age"

Neural network history

Second stage

- In 1982, the physicist Hopfield proposed a recursive network with associative memory and optimized computing power, i.e., the Hopfield network
- In 1986, Rumelhart et.al. published the book《Parallel Distributed Processing: Explorations in the Microstructures of Cognition》, in which the BP algorithm was reinvented
- In 1987, IEEE holds the first international conference on neural networks in San Diego, California（ICNN）
- In the early 1990s, statistical learning theory and SVM have emerged, while neural networks were suffering from the lacking of theories, heavily relying on trial-and-error and full of tricks.
- Neural networks enter another winter.

Neural network history

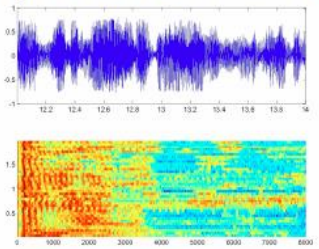
Third stage

- In 2006, Hinton proposed the Deep Belief Network (DBN), which makes the optimization of deep models relatively easy through "pre-training+fine-tuning"
- In 2012, the Hinton team participated in the ImageNet competition and won the championship of the year with a score of 10% over the second place using the CNN model
- With the advent of the cloud computing and big data era, computing power has greatly improved, making deep learning models have achieved great success in various fields

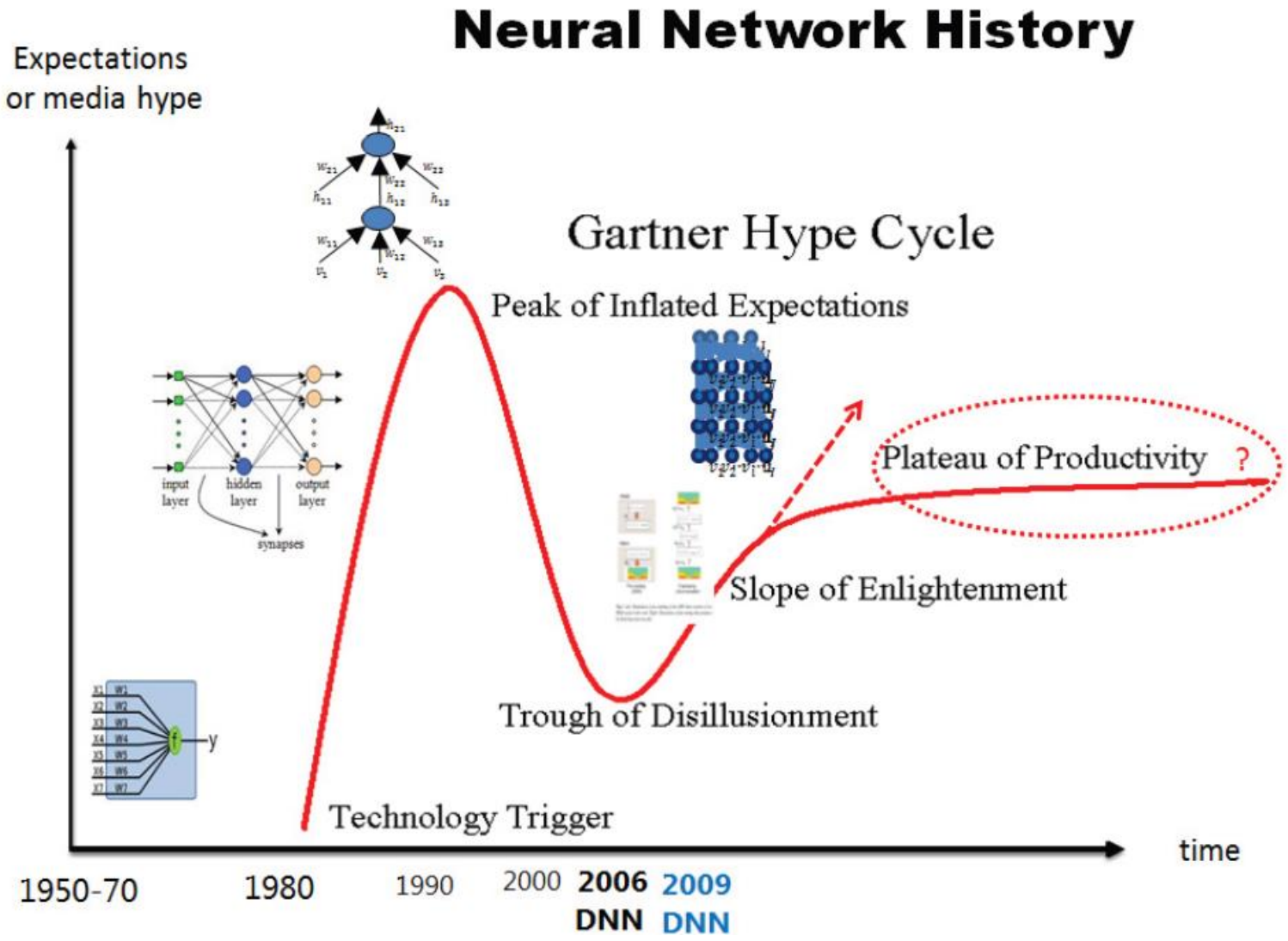
Images & Video

Text & Language

Speech & Audio



Neural network history



Introduction to Neural Networks

Outline of the paragraphs of the chapter that we will see

- 5.1 Neuron Model
- 5.2 Perceptron and Multi-layer Network
- 5.3 Error Backpropagation Algorithm

Ch05 Neural Networks

Outline

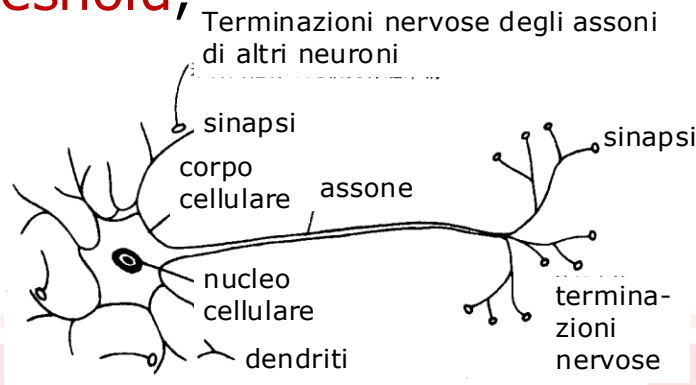
- 5.1 Neuron Model
- 5.2 Perceptron and Multi-layer Network
- 5.3 Error Backpropagation Algorithm

5.1 Neuron Model

Definitions of neural networks

"Neural networks are massively parallel interconnected networks of simple elements and their hierarchical organizations which are intended to interact with the objects of the real world in the same way as biological nervous systems do" [Kohonen, 1988]

- In the context of machine leaning, neural networks refer to "neural networks learning", or in other words, the intersection of machine learning research and neural networks research
- The basic element of neural networks is neuron, which is the "simple element" in the above definition
- Biological neural networks: the neurons, when "excited", send neurotransmitters to interconnected neurons to change their electric potentials.
  When the electric potential exceeds a threshold, the neuron is activated, and it will send neurotransmitters to other neurons.



5.1 Neuron Model

M-P Neuron Model [McCulloch and Pitts, 1943]

- Input: receive input signals from n neurons
- Process: The weighted sum of received signals is compared against the threshold
- Output: the output signal is produced by the activation function

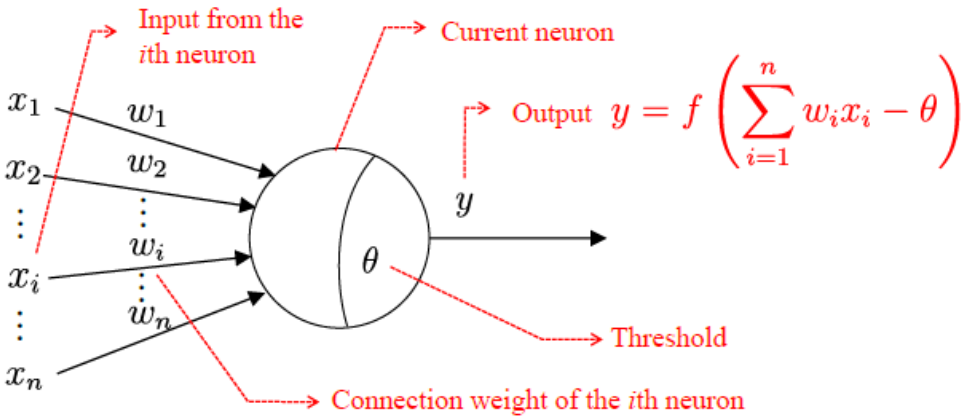


Fig. 5.1: The M-P neuron model.

5.1 Neuron Model

Activation Function

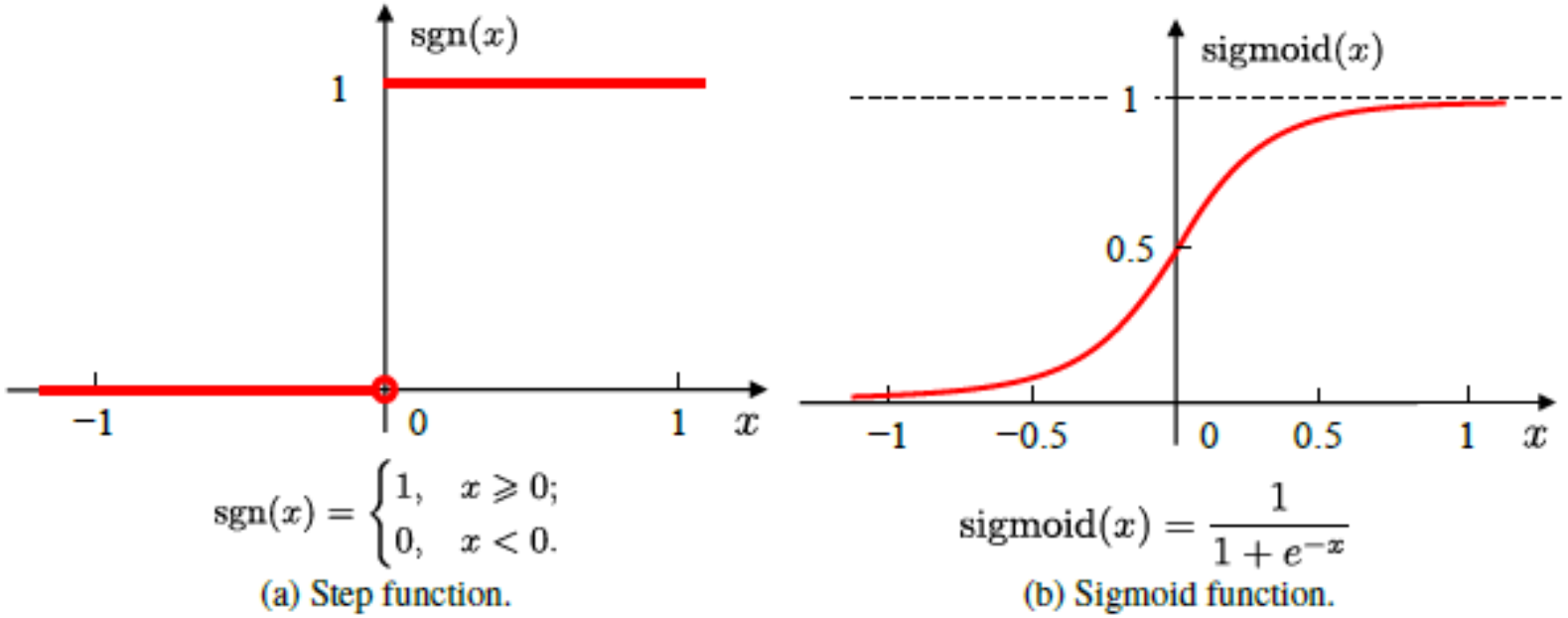


Fig. 5.2: Typical neuron activation functions.

- The ideal activation function is the step function, which maps the input value to “0” (non-excited) and “1” (excited).
- The step function are discontinuous and non-smooth, we often use the sigmoid function instead.

Ch05 Neural Networks

Outline

- 5.1 Neuron Model
- 5.2 Perceptron and Multi-layer Network
- 5.3 Error Backpropagation Algorithm
- 5.4 Global Minimum and Local Minimum
- 5.5 Other Common Neural Networks
- 5.6 Deep Learning

5.2 Perceptron and Multi-layer Network

Perceptron

- Perceptron is a binary classifier consisting of two layers of neurons. The input layer receives signals and transmits them to the output M-P neuron (threshold logic unit)
- Perceptron can easily implement "AND", "OR" and "NOT"
  - "AND" $x_1 \wedge x_2$: letting $w_1 = w_2 = 1, \theta = 2$, then $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$, and $y = 1$ iff $x_1 = x_2 = 1$
  - "OR" $x_1 \vee x_2$: letting $w_1 = w_2 = 1, \theta = 0.5$, then $y = f(1 \cdot x_1 + 1 \cdot x_2 - 0.5)$, and $y = 1$ iff $x_1 = 1$ or $x_2 = 1$.
  - "Not" $\neg x_1$: letting $w_1 = -0.6, w_2 = 0, \theta = -0.5$, then $y = 0$ when $x_1 = 1$ and $y = 1$ when $x_1 = 0$

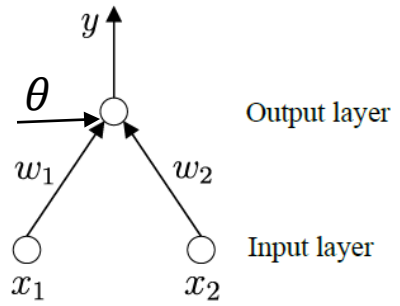


Fig. 5.3: A perceptron with two input neurons.

5.2 Perceptron and Multi-layer Network

Perceptron Learning

- The weight $w_i (i = 1, 2, \cdots, n)$ and threshold θ can be learned from training data.

- The learning rule

For training sample $(\boldsymbol{x}, y)$, if the output is $\hat{y}$, then the weight is updated by

$$w_i \leftarrow w_i + \triangle w_i$$
$$\triangle w_i = \eta (y - \hat{y}) x_i$$

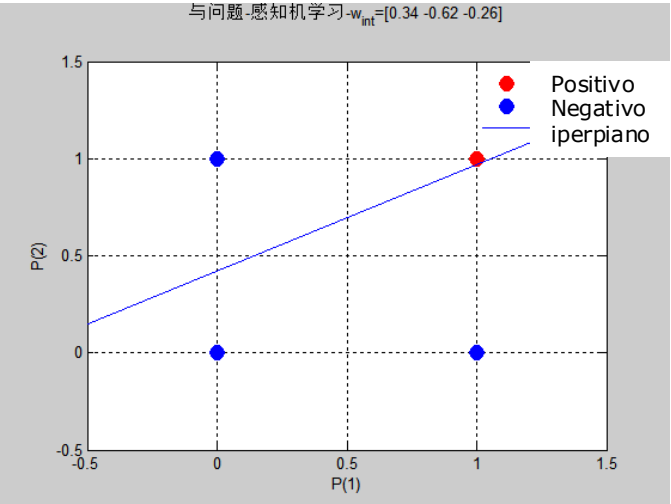
where $\eta \in (0, 1)$ is known as the learning rate.

The perceptron remains unchanged if it correctly predicts the sample $(\boldsymbol{x}, y)$; Otherwise, the weight is updated based on the degree of error.

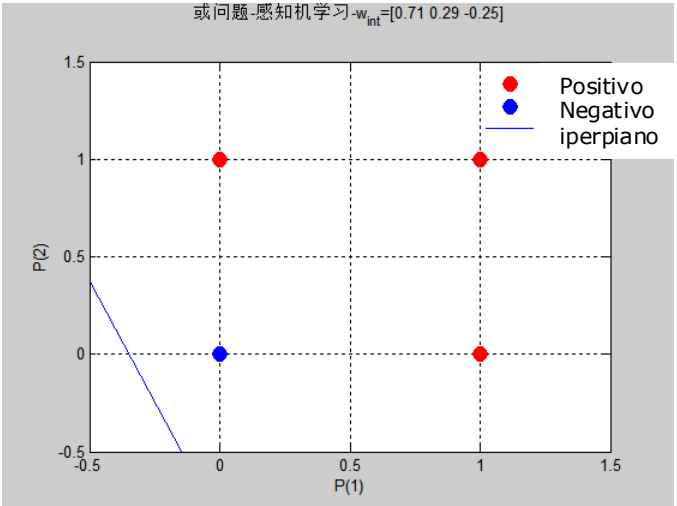
5.2 Perceptron and Multi-layer Network

Perceptron solving the “AND”, “OR” and “Not” problems

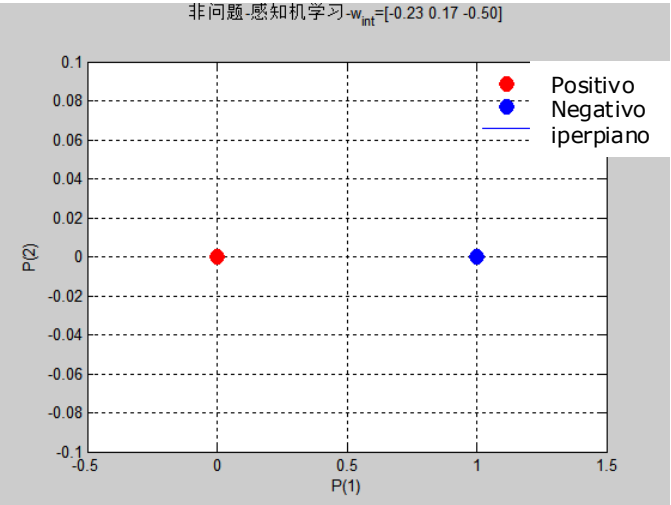
AND



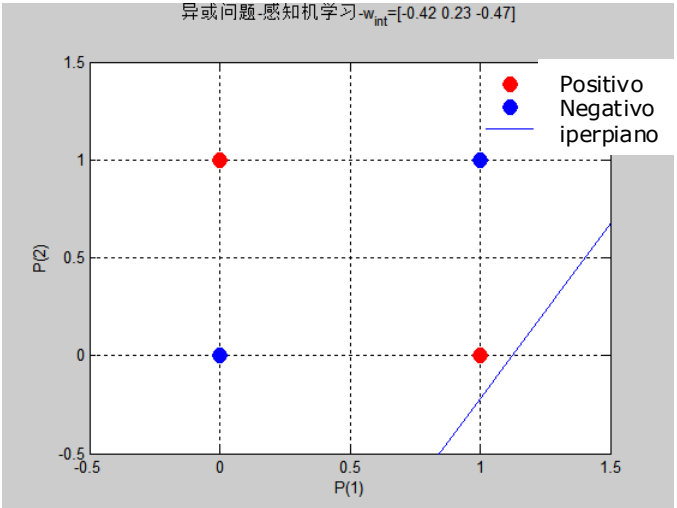
OR



NOT



XOR



5.2 Perceptron and Multi-layer Network

Perceptron solving the "AND", "OR" and "Not" problems

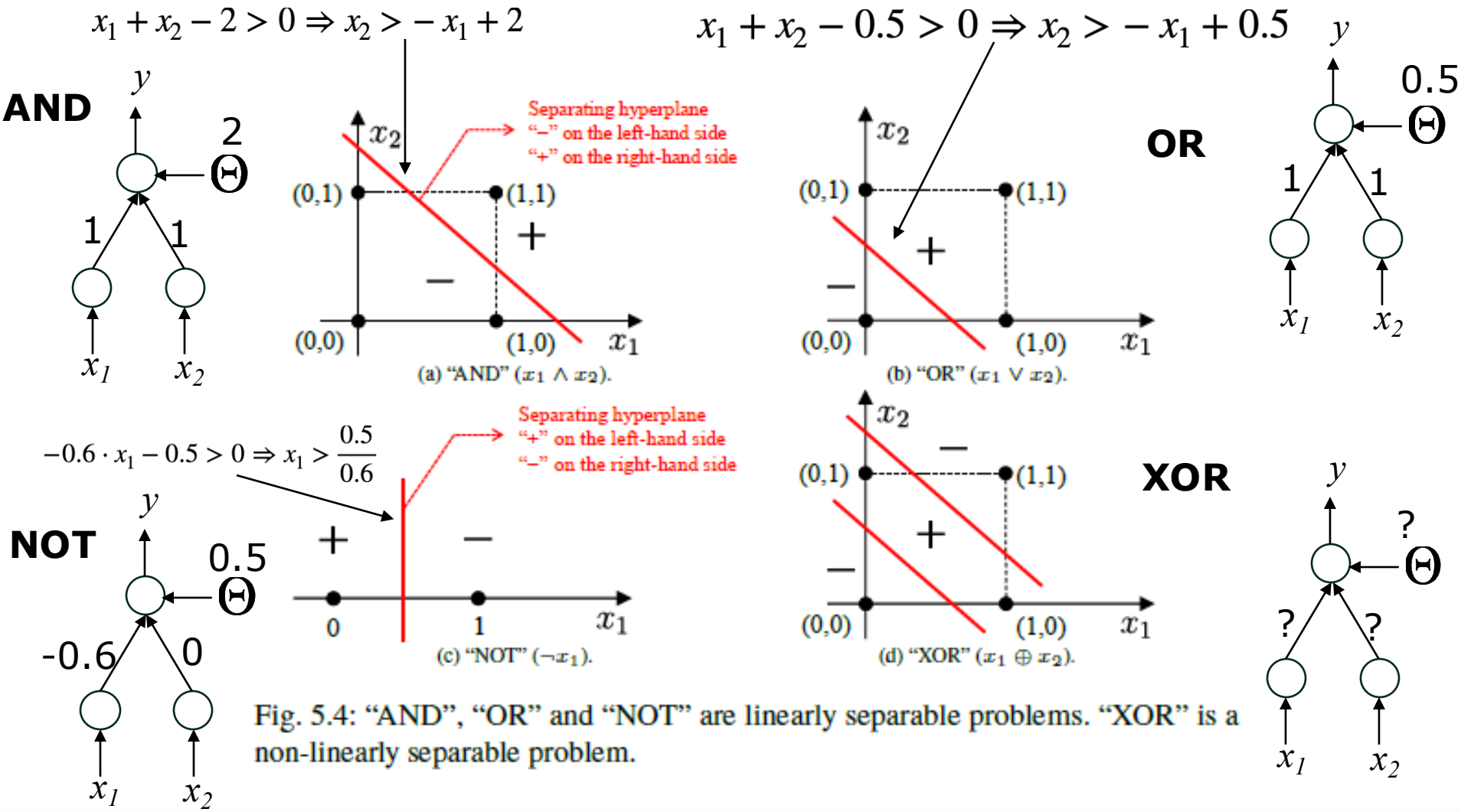


Fig. 5.4: "AND", "OR" and "NOT" are linearly separable problems. "XOR" is a non-linearly separable problem.

5.2 Perceptron and Multi-layer Network

The learning ability of perceptrons

- The learning process is guaranteed to converge for linear separable problems; otherwise fluctuation will happen.
  [Minsky and Papert, 1969]
- The learning ability of one-layer perceptrons is rather weak, which can only solve linear separable problems.

- In fact, the "AND", "OR" and "NOT" problems are all linear separable problems and the learning process is guaranteed to converge to an appropriate weight vector. However, perceptrons cannot solve non-linearly separable problems like "XOR".
- How to solve non-linearly separable problems ?

  Multi-layer perceptrons

5.2 Perceptron and Multi-layer Network

Multi-layer perceptrons

- A two-layer perceptron that solves the "XOR" problem.

$$b_1 + b_2 - 1.5 > 0 \Rightarrow b_1 + b_2 > 1.5 \Rightarrow x_1 \vee x_2 \text{ AND } \neg(x_1 \wedge x_2)$$

$$+x_2 + x_1 - 0.5 > 0 \Rightarrow x_2 > -x_1 + 0.5 \qquad -x_1 - x_2 + 1.5 > 0 \Rightarrow x_2 < -x_1 + 1.5$$

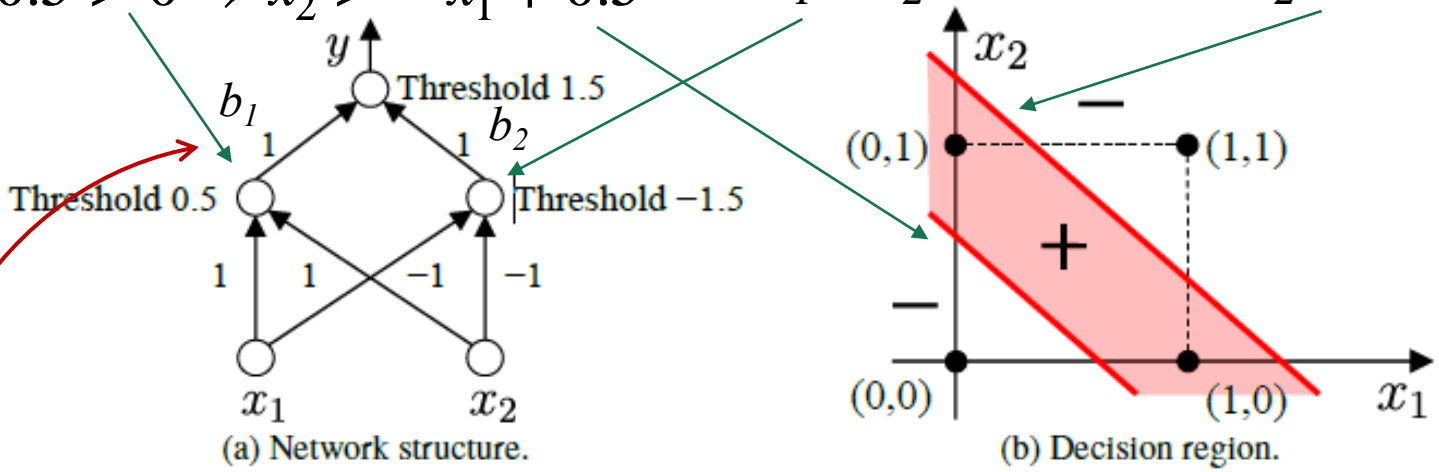


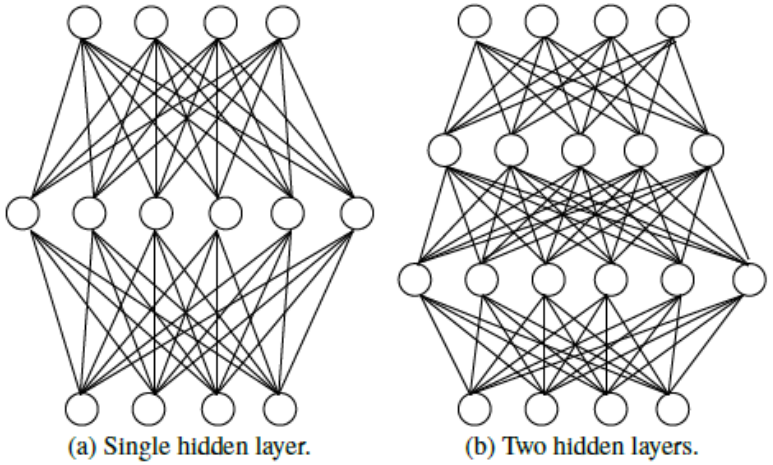
Fig. 5.5: A two-layer perceptron that solves the "XOR" problem.

- The neuron layer between the input and output layer is known as the hidden layer, which has activation functions as the output layer.

5.2 Perceptron and Multi-layer Network

Multi-layer feed-forward neural networks

- Definition: the neurons in each layer are fully connected with the neurons in the next layer.
- Forward: the input layer receives external signals, the hidden and output layers process (output) the signals.
- Learning: learning from data to adjust the "connection weights" and "thresholds".
- Multi-layer networks: neural networks with hidden layer(s)



(a) Single hidden layer. (b) Two hidden layers.

Fig. 5.6: Muti-layer feedforward neural networks.

Ch05 Neural Networks

Outline

- 5.1 Neuron Model
- 5.2 Perceptron and Multi-layer Network
- 5.3 Error Backpropagation Algorithm

5.3 Error Backpropagation Algorithm

Error BackPropagation algorithm (BP) is by far the most successful neural network learning algorithm.

- Given dataset $D = \{(\boldsymbol{x}_i, y_i)\}, \boldsymbol{x}_i \in R^d, y_i \in R^l, (i = 1, 2, \cdots, m)$
- For ease of discussion, given a multi-layer feedforward neural network with d input neurons, l output neurons and q hidden layers.
- Notations:

θ_j: threshold of the j-th output neuron;

γ_h: threshold of the h-th hidden neuron;

v_{ih}: connection weight between the i-th input neuron and h-th hidden layer neuron;

w_{hj}: connection weight between the h-th hidden layer neuron and the j-th output layer neuron;

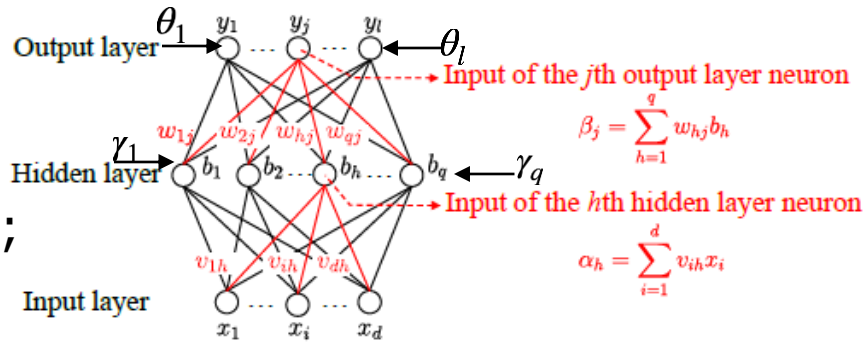


Fig. 5.7: Notations of BP neural networks.

5.3 Error Backpropagation Algorithm

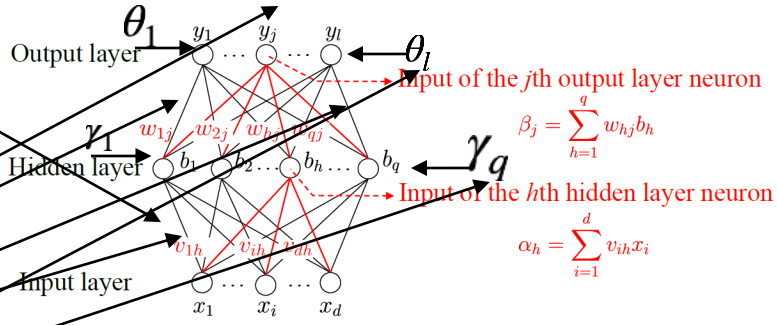
For sample $(\boldsymbol{x}_k, \boldsymbol{y}_k)$, suppose the network outputs $\hat{\boldsymbol{y}}_k$

- Forward

step$_1$: $b_h = f(\alpha_h - \gamma_h), \alpha_h = \Sigma_{i=1}^{d} v_{ih} x_i$

step$_2$: $\hat{y}_j^k = f(\beta_j - \theta_j); \beta_j = \Sigma_{h=1}^{q} w_{hj} b_h$

step$_3$: $E_k = \frac{1}{2} \sum_{j=1}^{l} (\hat{y}_j^k - y_j^k)^2$



- Number of parameters

weights: v_{ih}, w_{hj} threshold: $\theta_j, \gamma_h (i = 1, \cdots, d, h = 1, , \cdots, q, j = 1, \cdots, l)$

total $(d + l + 1)q + l$ parameters to be determined

- Parameter tuning

BP is an iterative learning algorithm, and each iteration employs the general form perceptron learning rule and the update rule for any parameter v is

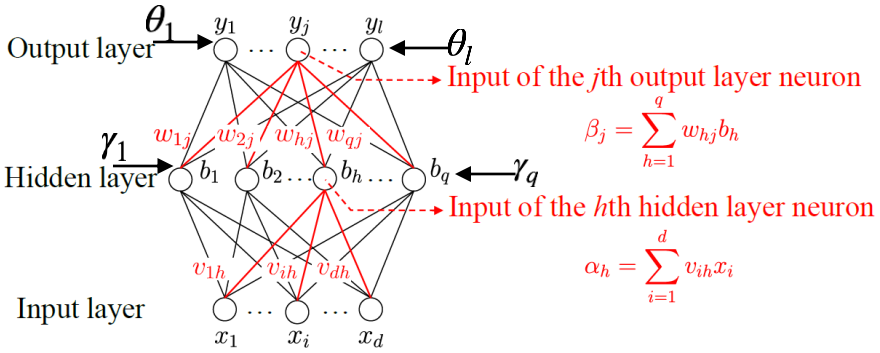
$$v \leftarrow v + \triangle v.$$

5.3 Error Backpropagation Algorithm

BP algorithm

- The BP algorithm employs gradient descent, which tunes the parameters towards the direction of the negative gradient of the objective. For error E_k and learning rate η, we have

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} \qquad E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$



5.3 Error Backpropagation Algorithm

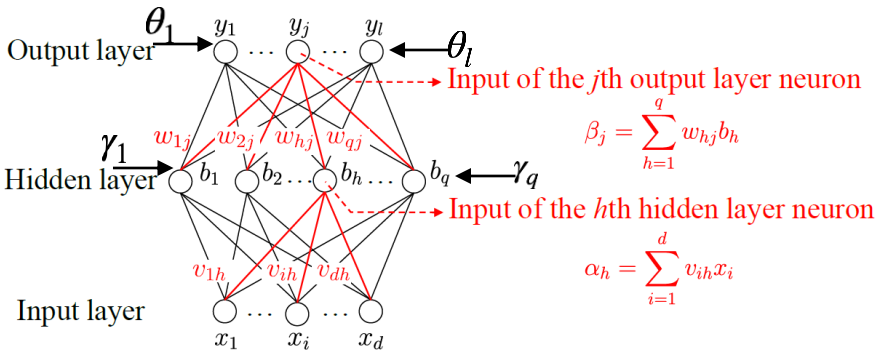
BP algorithm

- The BP algorithm employs gradient descent, which tunes the parameters towards the direction of the negative gradient of the objective. For error E_k and learning rate η, we have

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} \qquad E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$

$$-\frac{\partial E_k}{\partial w_{hj}} = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}} \qquad \hat{y}_j^k = f(\beta_j - \theta_j) \qquad \beta_j = \Sigma_{h=1}^{q} w_{hj} b_h$$

b_h



5.3 Error Backpropagation Algorithm

BP algorithm

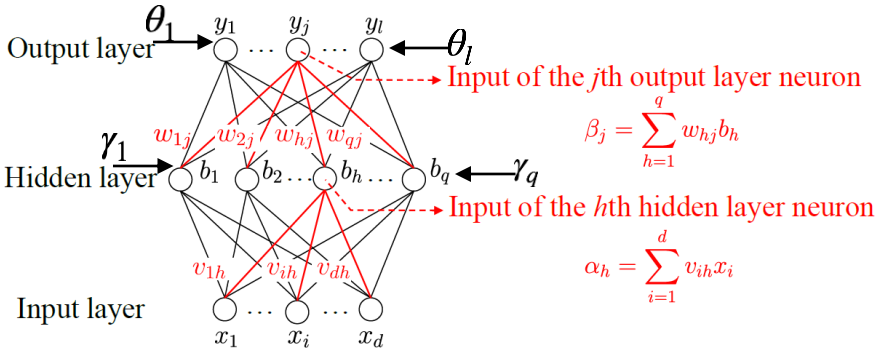
- The BP algorithm employs gradient descent, which tunes the parameters towards the direction of the negative gradient of the objective. For error E_k and learning rate η, we have

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} \qquad E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$

$$-\frac{\partial E_k}{\partial w_{hj}} = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}} \qquad \hat{y}_j^k = f(\beta_j - \theta_j) \qquad \beta_j = \Sigma_{h=1}^{q} w_{hj} b_h$$

$$g_j = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j}$$

b_h



5.3 Error Backpropagation Algorithm

BP algorithm

- The BP algorithm employs gradient descent, which tunes the parameters towards the direction of the negative gradient of the objective. For error E_k and learning rate η, we have

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} \qquad E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$

$$-\frac{\partial E_k}{\partial w_{hj}} = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}} \qquad \hat{y}_j^k = f(\beta_j - \theta_j) \qquad \beta_j = \Sigma_{h=1}^{q} w_{hj} b_h$$

$$g_j = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j}$$

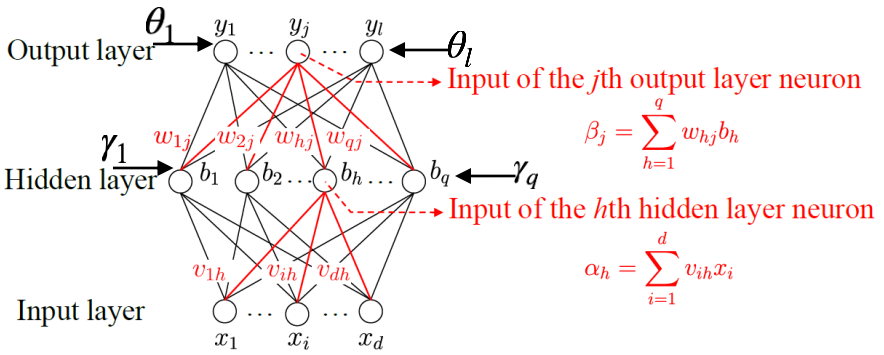
$$b_h$$

$$-\frac{2}{2}(\hat{y}_j^k - y_j^k)$$

$$f'(\beta_j - \theta_j)$$

f is sigmoid

$$f'(x) = f(x)[1 - f(x)]$$
$$= \hat{y}_j^k(1 - \hat{y}_j^k)$$



5.3 Error Backpropagation Algorithm (cont)

- Final computation:

$$g_j = (y_j^k - \hat{y}_j^k) \cdot \hat{y}_j^k \cdot (1 - \hat{y}_j^k) \qquad (5.10)$$

$$\Delta w_{hj} = \eta \cdot g_j \cdot b_h \qquad (5.11)$$

5.3 Error Backpropagation Algorithm

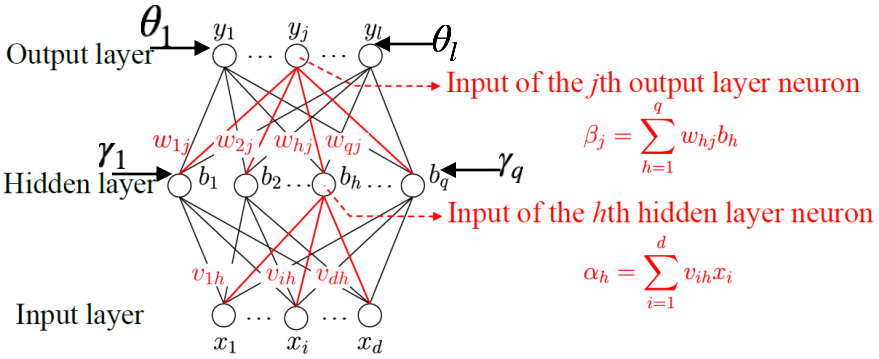
BP algorithm

- Similarly we can derive:

$$\triangle\theta_j = -\eta g_j, \quad (5.12)$$

$$\triangle v_{ih} = \eta e_h x_i, \quad (5.13)$$

$$\triangle\gamma_h = -\eta e_h, \quad (5.14)$$



where

$$e_h = -\frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h}$$

$$= -\sum_{j=1}^{l} \frac{\partial E_k}{\partial \beta_j} \frac{\partial \beta_j}{\partial b_h} f'(\alpha_h - \gamma_h) \qquad = b_h(1-b_h) \sum_{j=1}^{h} w_{hj} g_j. \quad (5.15)$$

$\beta_j = \Sigma_{h=1}^{q} w_{hj} b_h$

g_j

- The learning rate $\eta \in (0,1)$ controls the step size in each round.

A overly large learning rate may cause fluctuations while a too small value leads to slow convergence.

5.3 Error Backpropagation Algorithm

BP algorithm

Algorithm 5.1 Error BackPropagation.

Input: Training set $D = \{(x_k, y_k)\}_{k=1}^{m}$;
Learning rate η.

Process:

1: Randomly initialize connection weights and thresholds from $(0, 1)$
2: **repeat**
3: **for all** $(x_k, y_k) \in D$ **do**
4: Compute the output $\hat{y}_k$ for the current sample according to the current parameters and (5.3);
5: Compute the gradient term g_j of the output layer neuron according to (5.10);
6: Compute the gradient term e_h of the hidden layer neurons according to (5.15);
7: Update connection weights w_{hj}, v_{ih} and thresholds θ_j, γ_h according to (5.11)–(5.14).
8: **end for**
9: **until** The termination condition is met

Output: A feedforward neural network with determined connection weights and thresholds.

$$g_j = (y_j^k - \hat{y}_j^k) \cdot \hat{y}_j^k \cdot (1 - \hat{y}_j^k)$$

$$\Delta w_{hj} = \eta \cdot g_j \cdot b_h$$

$$\triangle \theta_j = -\eta g_j,$$

$$\triangle v_{ih} = \eta e_h x_i,$$

$$\triangle \gamma_h = -\eta e_h,$$

$$e_h = b_h \cdot (1 - b_h) \Sigma_j w_{hj} \cdot g_j$$

5.3 Error Backpropagation Algorithm

BP algorithm experiments

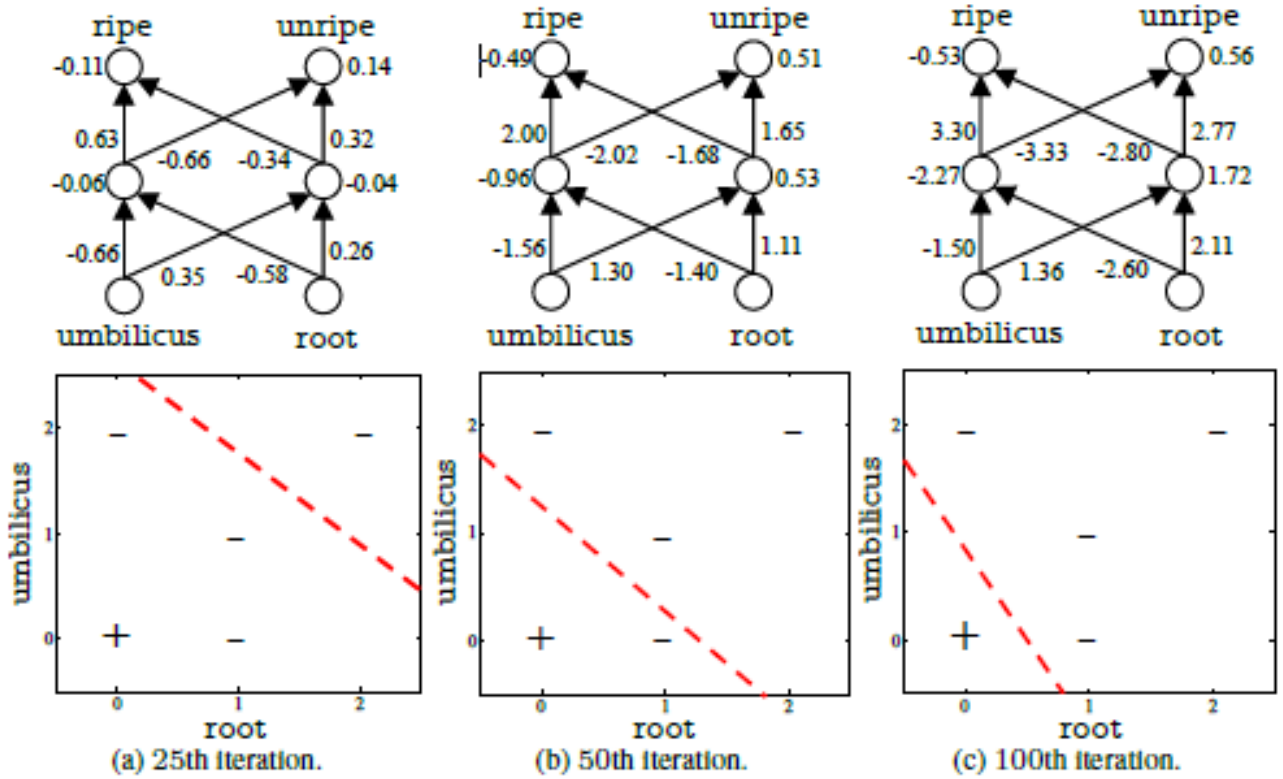


Fig. 5.8: The changes of parameters and decision boundaries of a BP neural network running on a watermelon data set with 5 samples and 2 features.

5.3 Error Backpropagation Algorithm

Standard BP algorithm

- uses one sample at a time to update weights and thresholds.
- updates frequently, updates of different samples may "offset" each other, needs more iterations.

Accumulated BP algorithm

- minimizes the accumulated error on the whole training set

$$E = \frac{1}{m}\sum_{k=1}^{m} E_k$$

- tunes parameters less frequently since it tunes once after a full scan of training set.

Practical applications

In some tasks, especially when the training set is large, the accumulated BP algorithm can become slow. In contrast, the standard BP algorithm can achieve a good solution quicker.

5.3 Error Backpropagation Algorithm

Representation ability of multi-layer networks

a feedforward neural network consisting of a single hidden layer with sufficient neurons could approximate continuous functions of any complexity up to arbitrary accuracy **[Hornik et al. , 1989]**

Limitations of multi-layer networks

- BP neural networks suffer from over-fitting: the training error decreases while the testing error increases.
- There is no principled method for setting the number of hidden layer neurons, and trial-by-error is usually used in practice.

Strategies to alleviate the overfitting problem

- Early stopping: Once the training error decreases while the validation error increases, the training process stops.
- Regularization: add a regularization term to the objective function, describing the complexity of neural network.

Reading Material

Mainstream academic journals

- Neural Computation
- Neural Networks
- IEEE Transactions on Neural Networks and Learning Systems

International conferences

- Advances in Neural Information Processing Systems（NIPS）
- International Joint Conference on Neural Network（IJCNN）

Regional international conferences

- International Conference on Artificial Neural Networks（ICANN）
- International Conference on Neural Information Processing（ICONIP）

Thanks!